

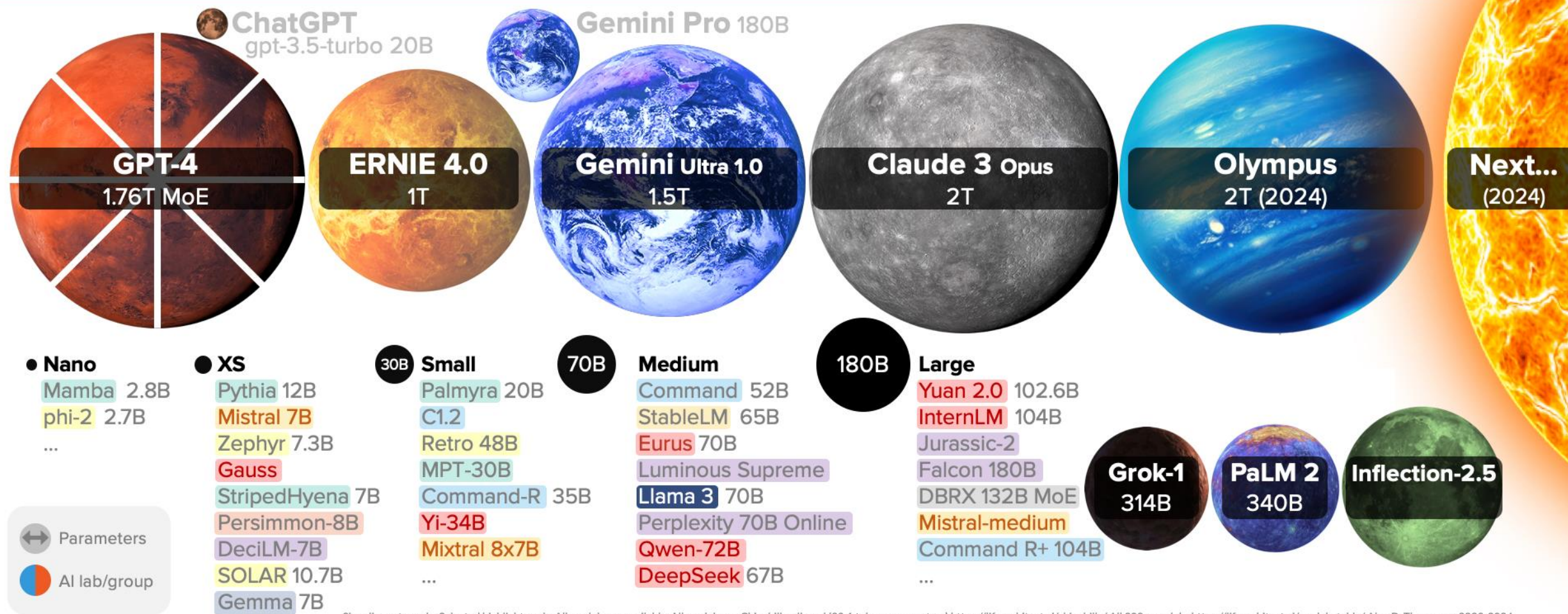


Estimating LLMs' Resource Requirement



Mohammad Javad Dousti

LARGE LANGUAGE MODEL HIGHLIGHTS (APR/2024)



Sizes linear to scale. Selected highlights only. All models are available. All models are Chinchilla-aligned (20:1 tokens:parameters) <https://lilearchitect.ai/chinchilla/> All 300+ models: <https://lilearchitect.ai/models-table/> Alan D. Thompson, 2023-2024.

Computation Budget

- Suppose, C is the compute required to train a transformer model, in total floating point operations:

$$C \approx \tau T$$

- τ is the aggregate throughput of your hardware setup

- $\tau = \# \text{GPUs} \times \frac{\text{Actual FLOPs}}{\text{GPU}}$ in FLOPs

- T is the time spent training the model (in seconds)

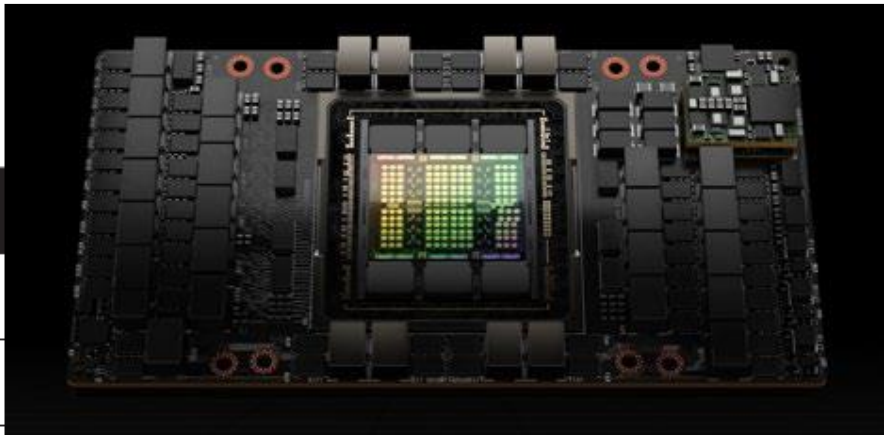
- What's the unit of C ?

- FLOP-seconds, which is in units of $\left[\frac{\text{Floating Point Operations}}{\text{Second}} \right] \times [\text{Seconds}]$
 - GPU-hours, which is in units of $[\# \text{GPUs}] \times [\text{Hours}]$
 - Scaling laws papers tend to report values in PetaFLOP-days, or $10^{15} \times 24 \times 3600$ total floating point operations

NVIDIA H100 Performance

Specifications		
	H100 SXM	H100 PCIe
FP64	34 TFLOPS	26 TFLOPS
FP64 Tensor Core	67 TFLOPS	51 TFLOPS
FP32	67 TFLOPS	51 TFLOPS
TF32 Tensor Core	989 TFLOPS*	756 TFLOPS*
BFLOAT16 Tensor Core	1,979 TFLOPS*	1,513 TFLOPS*
FP16 Tensor Core	1,979 TFLOPS*	1,513 TFLOPS*
FP8 Tensor Core	3,958 TFLOPS*	3,026 TFLOPS*
INT8 Tensor Core	3,958 TOPS*	3,026 TOPS*
GPU memory	80GB	80GB
GPU memory bandwidth	3.35TB/s	2TB/s

* Shown with sparsity. Specifications 1/2 lower without sparsity.



Source: NVIDIA H100 Tensor Core GPU datasheet

Computation Budget (cont'd)

$$C \approx \tau T = 6ND$$

□ We can write C as follows:

➤ $C = C_{\text{forward}} + C_{\text{backward}}$

○ $C_{\text{forward}} = 2ND$

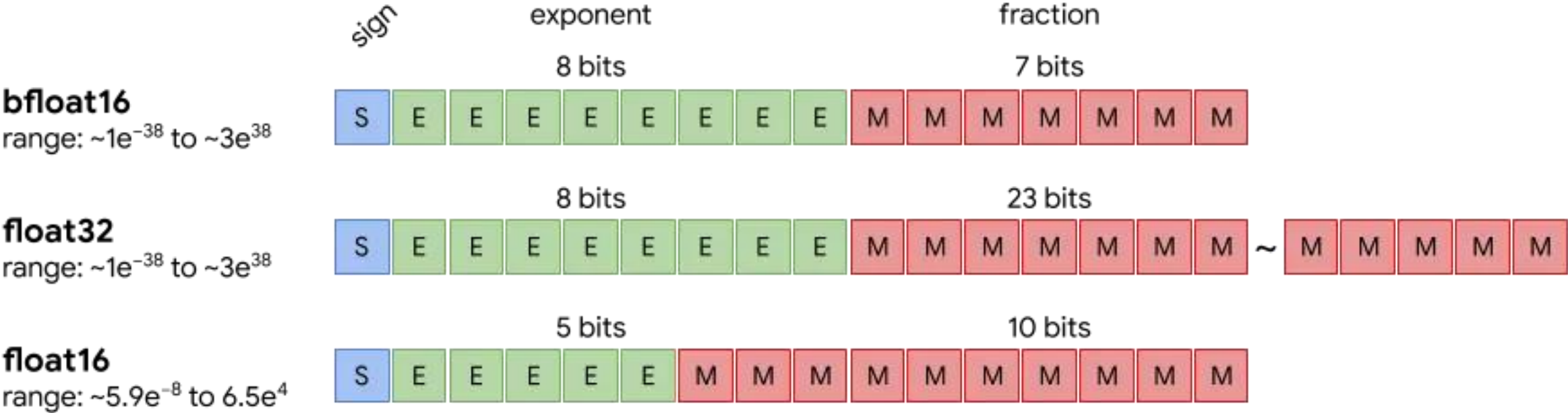
○ $C_{\text{backward}} = 4ND$

○ N is the number of parameters in the transformer model

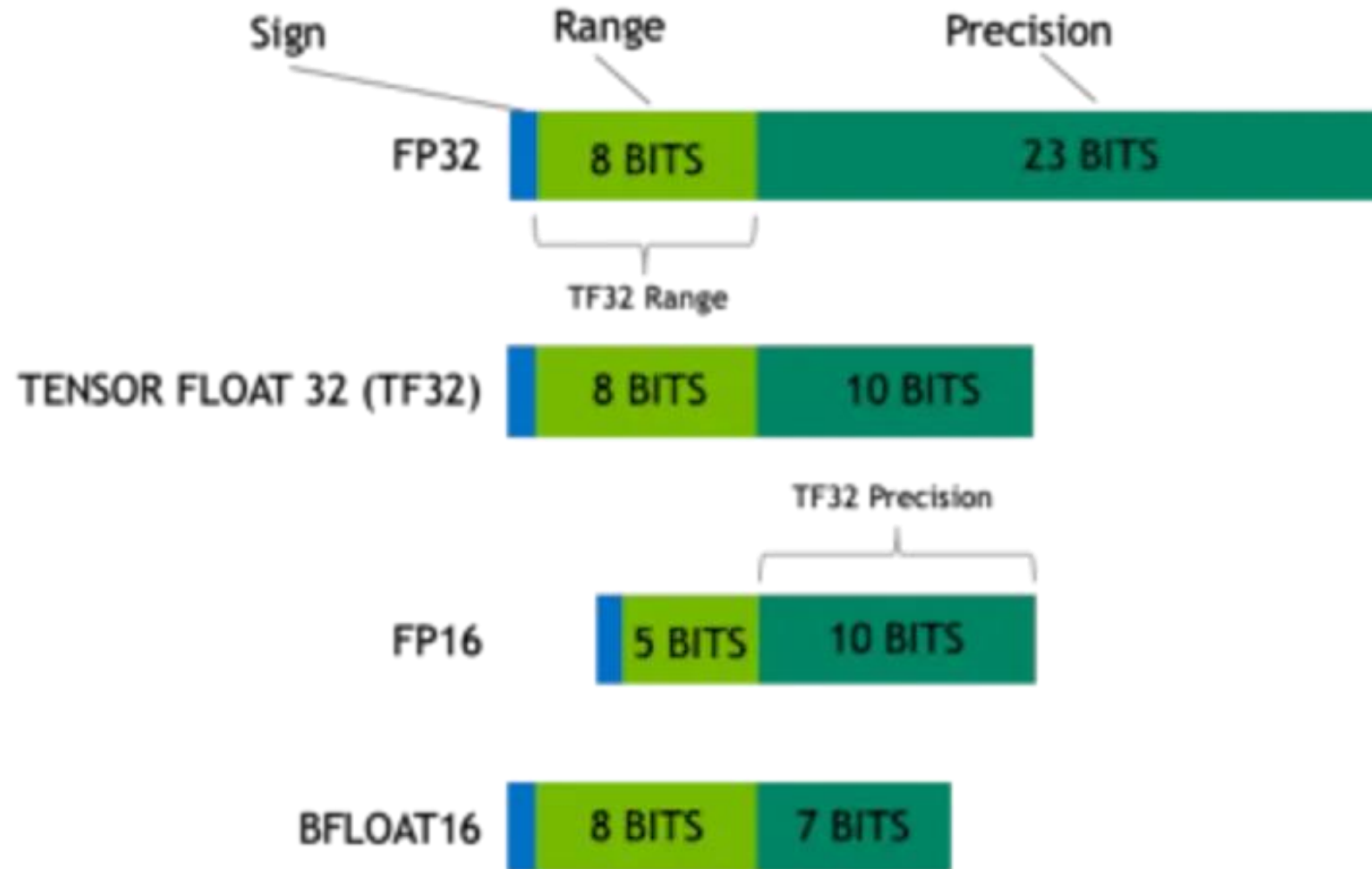
○ D is the dataset size (in tokens)

Large Models' Memory Requirements

GPU Datatypes: Floating Point



GPU Datatypes: Floating Point (cont'd)



GPU Datatypes:

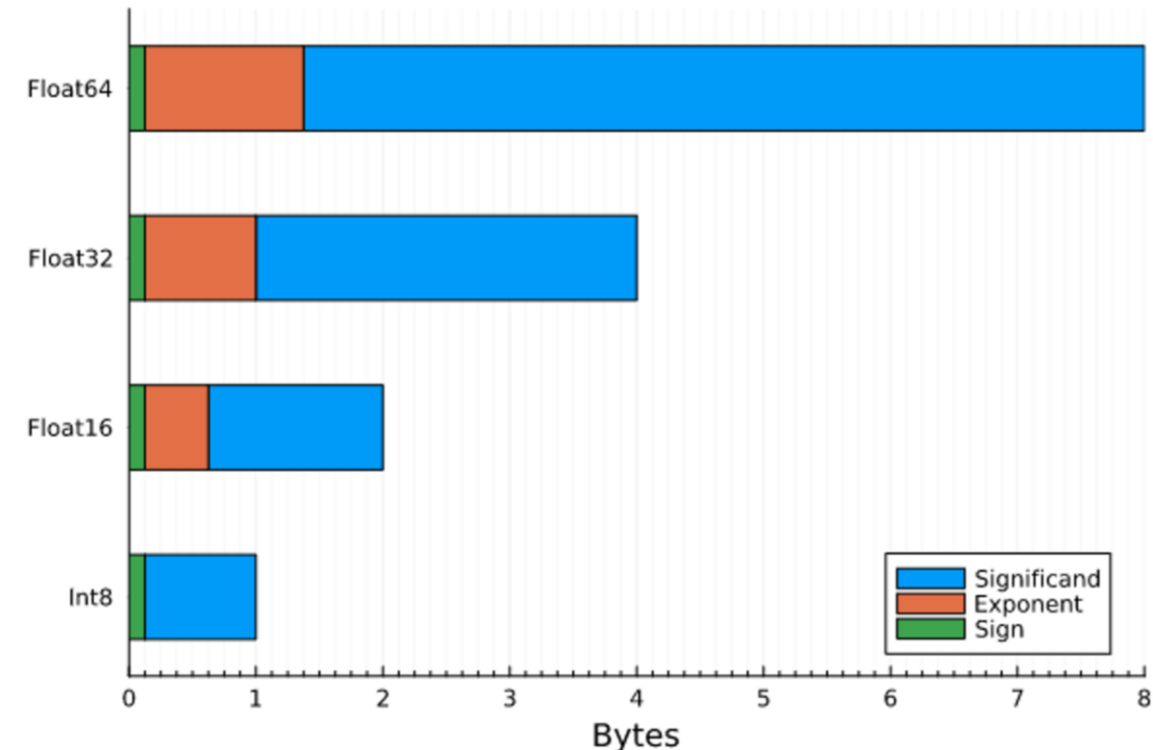
GPU Architecture	fp32	fp16	bf16	fp8	int8	int4
Volta	Y	Y	N	N	Y	Y
Turing	Y	Y	N	N	Y	Y
Ampere	Y	Y	Y	N	Y	Y
Ada-Lovelace	Y	Y	Y	Y	Y	Y
Hopper	Y	Y	Y	Y	Y	Y

- ❖ Volta (datacenter): V100
- ❖ Turing (consumer): 2000-series
- ❖ Ampere (consumer/datacenter): 3000-series & A100
- ❖ Ada-Lovelace (consumer/datacenter): 4000-series
- ❖ Hopper (datacenter): H100 and H200

Inference Memory Requirement

□ Memory to store model parameters:

- `int8`: (1 byte / param)(# params)
- `fp16/bf16`: (2 bytes / param)(# params)
- `fp32`: (4 bytes / param)(# params)



□ Total inference memory:

$$\text{Total Memory}_{\text{inference}} \approx \underbrace{1.2}_{\text{overhead}} \times \text{memory}_{\text{model}}$$

Training Memory Requirement

$$\text{Total mem}_{\text{training}} = \text{mem}_{\text{model}} + \text{mem}_{\text{optimizer}} + \text{mem}_{\text{gradients}} + \text{mem}_{\text{activations}}$$

□ Model memory requirement ($\text{mem}_{\text{model}}$):

- Pure **fp32**: $\text{mem}_{\text{model}} = (4 \text{ bytes} / \text{param})(\# \text{ params})$
- Pure **fp16/bf16**: $\text{mem}_{\text{model}} = (2 \text{ bytes} / \text{param})(\# \text{ params})$
- Mixed precision training (**fp16/bf16** and **fp32**):
 - fp32 is stable but takes a lot of memory. It isn't supported by tensor cores.
 - fp16 is stable but difficult to converge. It enables us to use tensor cores.
 - Mixed precision training retains two copies of a model. One in fp16/bf16 and another in fp32.

$$(2 \text{ bytes} / \text{param})(\# \text{ params}) + (4 \text{ bytes} / \text{param})(\# \text{ params})$$

This term will be counted as a part of optimizer state

Optimizer States Memory ($\text{mem}_{\text{optimizer}}$)

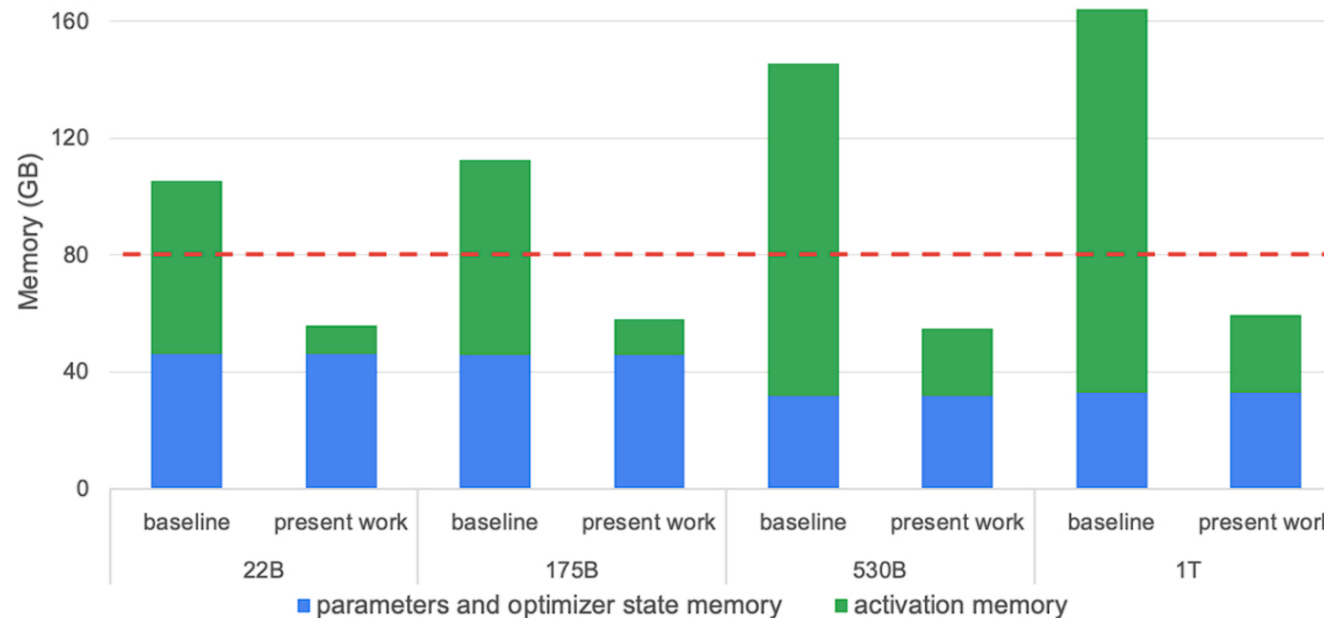
- ❑ Vanilla AdamW optimizer: (12 bytes/param)(# params)
 - fp32 copy of parameters: 4 bytes/param
 - Momentum: 4 bytes/param
 - Variance: 4 bytes/param
- ❑ SGD-like optimizers with momentum: (8 bytes/param)(# params)
 - fp32 copy of parameters: 4 bytes/param
 - Momentum: 4 bytes/param
- ❑ 8-bit optimizers (e.g., bitsandbytes): (6 bytes/param)(# params)
 - fp32 copy of parameters: 4 bytes/param
 - Momentum: 1 byte/param
 - Variance: 1 byte/param

Gradients Memory ($\text{mem}_{\text{gradients}}$)

- ❑ Gradients can be stored in **fp32** or **fp16**.
 - Usually they follow the model datatype.
- ❑ **fp32** training: (4 bytes/param)(# params)
- ❑ **fp16/mixed-precision training**: (2 bytes/param)(# params)

Activations Memory ($\text{mem}_{\text{activations}}$)

- ❑ Modern GPUs are typically bottlenecked by memory, not FLOPs, for LLM training.
- ❑ Activation recomputation/checkpointing is a very popular method of trading reduced memory costs for extra compute costs.



V.A. Korthikanti et al, "[Reducing activation recomputation in large transformer models](#)," *Proceedings of Machine Learning and Systems 5* (2023).

Activations Memory ($\text{mem}_{\text{activations}}$) – cont'd

$$\text{mem}_{\text{activations}}^{\text{no recomputation}} = sbhL \left(10 + \frac{24}{t} + 5 \frac{a.s}{h.t} \right) \text{ bytes}$$

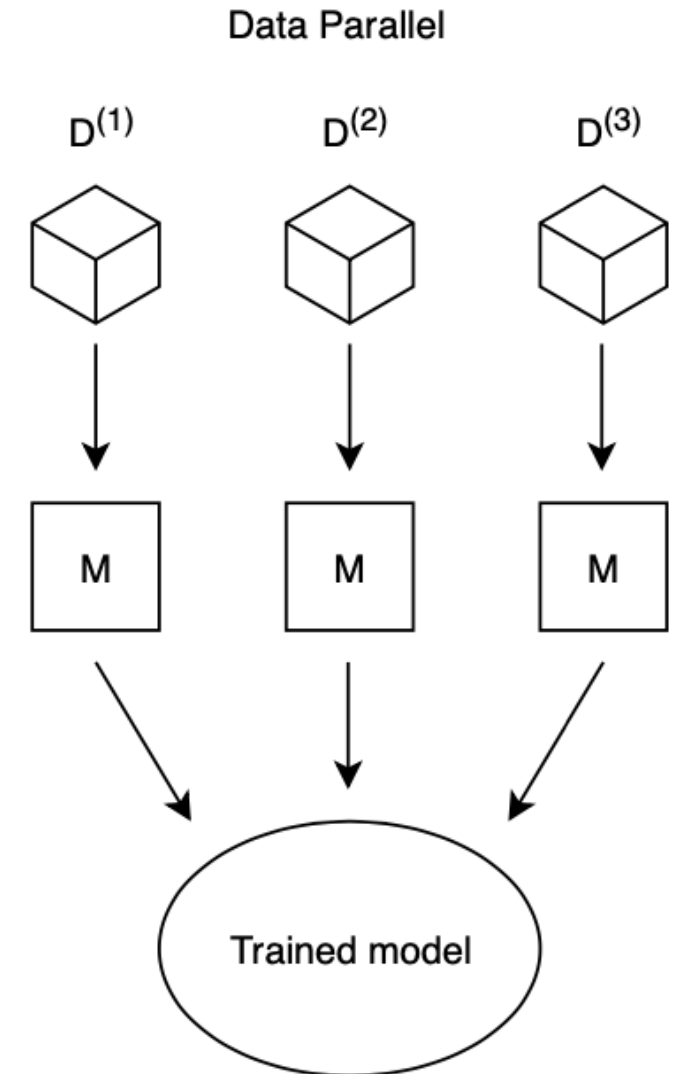
$$\text{mem}_{\text{activations}}^{\text{selective recomputation}} = sbhL \left(10 + \frac{24}{t} \right) \text{ bytes}$$

$$\text{mem}_{\text{activations}}^{\text{full recomputation}} = sbhL \text{ bytes}$$

- s is the sequence length (in tokens)
 - b is the batch size per GPU
 - h is the dimension of the hidden size within each transformer layer
 - L is the number of layers in the transformer model
 - a is the number of attention heads in the transformer model
 - t is the degree of tensor parallelism being used (1 if not)
- Additional recomputation may increase C_{forward} by at most $2PD$. Why?

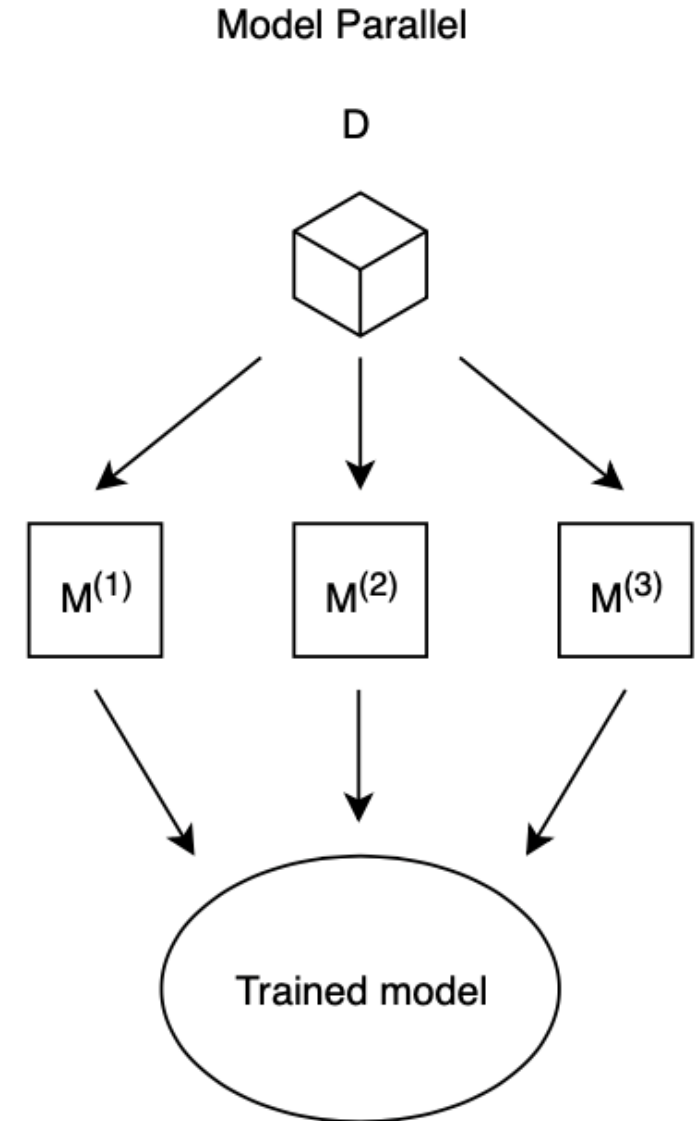
Data Parallel Approach

- ❑ Data are partitioned among n worker nodes.
- ❑ The same model is available to all worker nodes.
- ❑ After that, gradients are calculated for each copy of the model, and then **all the models exchange the gradients**, which are **subsequently averaged**.
- ❑ Can be used by **most ML algorithm** which assumes **independent and identically distribution (i.i.d.)** over the data samples.
 - Can you name an algorithm which doesn't make such assumption?
 - *Curriculum learning* is a good example where data parallelism doesn't work.



Model Parallelism Approach

- ❑ Certain models are so big that cannot fit in a single node's memory (e.g., GPU memory).
- ❑ Model parallelism:
 - Model is divided among all nodes.
 - Each nodes processes an identical copy of the dataset.
- ❑ Model parallelism is quite fast for large networks and very slow for small networks, while the opposite is true for data parallelism.
- ❑ Model parallelism has lower communication overhead compared to data parallelism, but there's no trick to hide communication required for synchronization of a network.



Model Parallelism Example

- ❑ Each LSTM layer is assigned to a different GPU.
 - After GPU 1 finishes computing layer 1 for the first sentence, it passes its output to GPU 2.
 - At the same time, GPU 1 fetches the next sentence and starts training.
- ❑ Is this really *parallel*?
 - Maybe it should have been called “*Model Serialization*” or “*Model Pipelining*.”

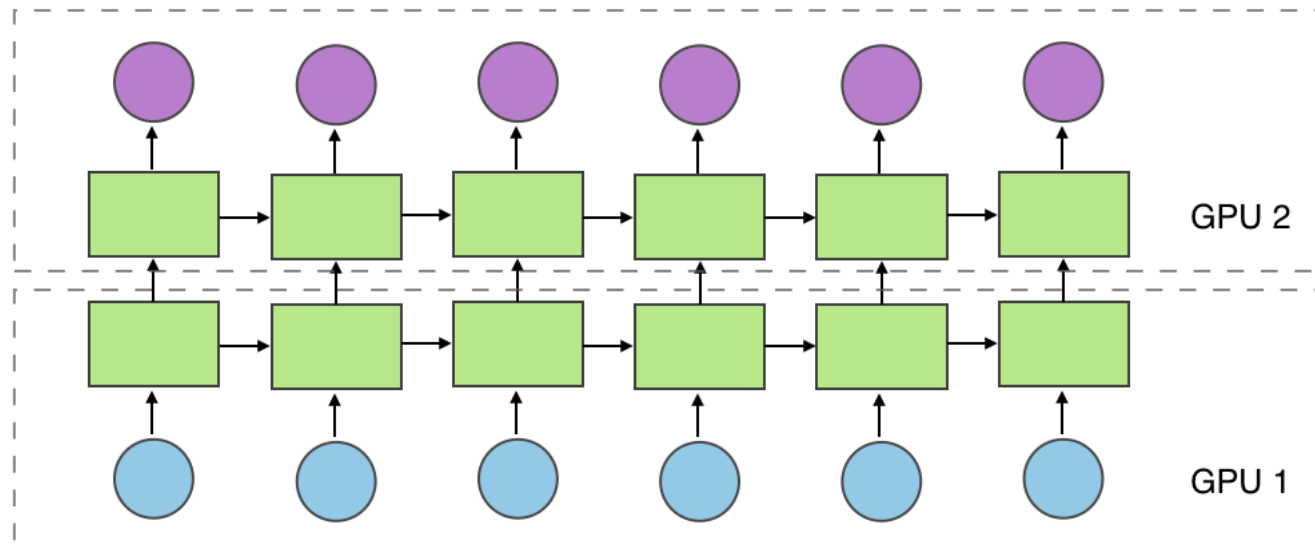


Image source: https://mxnet.incubator.apache.org/versions/1.8.0/api/faq/model_parallel_lstm.html